

Visão geral da Linguagem C

Estrutura de Dados — QXD0010



UNIVERSIDADE
FEDERAL DO CEARÁ
CAMPUS QUIXADÁ

Prof. Atílio Gomes Luiz
gomes.atilio@ufc.br

Universidade Federal do Ceará

2º semestre/2025



Tópicos

- Compilação e execução
- Elementos básicos da linguagem C
- Estruturas de seleção e repetição
- Vetores
- Structs e enumerações
- Ponteiros
- Alocação dinâmica de memória
- Ponteiros para ponteiros
- Arrays de caracteres



Tutoriais online para estudo rápido e consulta

- learn-c.org: Um tutorial interativo gratuito e sem necessidade de instalação: você escolhe o capítulo e já pode programar no navegador
- [W3Schools C Tutorial](https://www.w3schools.com/c/): Plataforma gratuita com editor, você modifica o código e executa direto no site
- [FreeCodeCamp](https://www.freecodecamp.org/): Apresenta uma abordagem direta e eficiente com os principais conceitos da linguagem

Apresentação da Linguagem

- A linguagem C é considerada uma linguagem de programação **imperativa** e **procedural**
 - **Imperativa**: O programador descreve passo a passo como o computador deve executar as instruções
 - **Procedural**: O código é organizado em funções (procedimentos), que podem ser reutilizadas. Segue os princípios da **programação estruturada** (sequência, seleção e repetição).

Apresentação da Linguagem

- A linguagem C é considerada uma linguagem de programação **imperativa** e **procedural**
 - **Imperativa**: O programador descreve passo a passo como o computador deve executar as instruções
 - **Procedural**: O código é organizado em funções (procedimentos), que podem ser reutilizadas. Segue os princípios da **programação estruturada** (sequência, seleção e repetição).

Foi desenvolvida por Dennis Ritchie (com contribuições de Ken Thompson) nos Laboratórios Bell, nos anos 1970. É uma das linguagens mais influentes da história da computação.



Apresentação da Linguagem

Padronizações da linguagem:

- K&R C (1978, não oficial)
- ANSI C (C89, 1989)
- ISO C (C90, 1990)
- C95 (1995)
- C99 (1999)
- **C11 (2011)**
- C17/C18 (2017/2018)
- C23 (2023/2024)

C possui diversas bibliotecas

- assert.h
- ctype.h
- errno.h
- float.h
- limits.h
- math.h
- stdlib.h
- string.h
- time.h
- stddef.h
- iso646.h
- locale.h
- setjmp.h
- signal.h
- stdarg.h
- stdbool.h
- stdint.h
- stdio.h
- uchar.h
- wchar.h
- wctype.h

Primeiro Programa — Hello World

```
1 #include <stdio.h>
2
3 int main() {
4     printf("Hello world!\n");
5     return 0;
6 }
```


Primeiro Programa — Hello World

```
1 #include <stdio.h>
2
3 int main() {
4     printf("Hello world!\n");
5     return 0;
6 }
```

Supondo que o programa acima esteja no arquivo `programa.cpp`, para compilar no terminal do Linux e depois executar:

- `$ gcc -Wall -Wextra -pedantic-errors programa.c -o main`
- `$ ./main`
 - `-Wall` é a abreviação de “warn all” – ativa (quase) todos os avisos que o `g++` pode lhe informar.
 - `-Wextra` ativa alguns avisos extras que não são ativados por `-Wall`
 - `-pedantic-errors` rejeita programas que não seguem o padrão ISO
 - `-o` altera o nome do arquivo de saída.

Elementos básicos da linguagem C



Elementos básicos da linguagem

Como em outras linguagens:

- Comentários de código
- Variáveis e Constantes
- Identificadores
- Tipos Fundamentais
- Representação numérica
- Vetores, strings, ponteiros, estruturas e enumerações
- Estruturas de controle de fluxo
- Funções
- entre outras...

Comentários

- Um **comentário** é uma descrição inserida diretamente no código-fonte do programa e que é ignorada pelo compilador. Serve apenas para uso do programador.
- C permite fazer comentários de duas maneiras diferentes:
por linha ou **por bloco**.

```
1 #include <stdio.h>
2
3 int main() {
4     /* --- Exemplo de comentario em bloco ---
5     A funcao printf
6     serve para
7     escrever na tela
8     */
9     printf("Hello World\n"); // Um comentario em linha
10
11     return 0;
12 }
```

Variáveis



Variáveis

- Em computação, uma **variável** é uma posição de memória onde poderemos guardar determinado dado ou valor e modificá-lo ao longo da execução do programa.

Variáveis

- Em computação, uma **variável** é uma posição de memória onde poderemos guardar determinado dado ou valor e modificá-lo ao longo da execução do programa.
- Quando criamos uma variável e armazenamos um valor dentro dela, o computador reserva um espaço associado a um endereço de memória onde podemos guardar o valor dessa variável.

Variáveis

- Em computação, uma **variável** é uma posição de memória onde poderemos guardar determinado dado ou valor e modificá-lo ao longo da execução do programa.
- Quando criamos uma variável e armazenamos um valor dentro dela, o computador reserva um espaço associado a um endereço de memória onde podemos guardar o valor dessa variável.
- O nome da variável é chamado **identificador**.

Variáveis

- Em computação, uma **variável** é uma posição de memória onde poderemos guardar determinado dado ou valor e modificá-lo ao longo da execução do programa.
- Quando criamos uma variável e armazenamos um valor dentro dela, o computador reserva um espaço associado a um endereço de memória onde podemos guardar o valor dessa variável.
- O nome da variável é chamado **identificador**.

```
1 #include <stdio.h>
2
3 int main() {
4     int x; //declara a variavel mas nao define o valor
5     printf("x = %d\n", x);
6     x = 5; //define o valor de x como sendo 5
7     printf("x = %d\n", x);
8     return 0;
9 }
```

Inicialização de variáveis escalares

```
1 #include <stdio.h> // prog07.c
2
3 int main() {
4     int z = 23; // por atribuicao -- assignment
5
6     printf("Valor de z: %d\n", z);
7
8     int k = 45.89; // ok, mas com perda de precisao
9
10    printf("Valor de k: %d\n", k);
11
12    return 0;
13 }
```

Identificadores

A linguagem C estipula algumas regras para a escolha dos identificadores:

- Um **identificador** é um conjunto de caracteres que podem ser letras, números ou *underscores* (`_`).

Identificadores

A linguagem C estipula algumas regras para a escolha dos identificadores:

- Um **identificador** é um conjunto de caracteres que podem ser letras, números ou *underscores* (-).
- O identificador deve sempre iniciar com uma letra ou o *underscore* (-).

Identificadores

A linguagem C estipula algumas regras para a escolha dos identificadores:

- Um **identificador** é um conjunto de caracteres que podem ser letras, números ou *underscores* (`_`).
- O identificador deve sempre iniciar com uma letra ou o *underscore* (`_`).
- A linguagem C é **case-sensitive**, ou seja, uma palavra escrita utilizando caracteres maiúsculos é diferente da mesma palavra escrita com caracteres minúsculos.

Identificadores

A linguagem C estipula algumas regras para a escolha dos identificadores:

- Um **identificador** é um conjunto de caracteres que podem ser letras, números ou *underscores* (`_`).
- O identificador deve sempre iniciar com uma letra ou o *underscore* (`_`).
- A linguagem C é **case-sensitive**, ou seja, uma palavra escrita utilizando caracteres maiúsculos é diferente da mesma palavra escrita com caracteres minúsculos.
- Palavras reservadas não podem ser usadas como nome de variáveis.

Identificadores

A linguagem C estipula algumas regras para a escolha dos identificadores:

- Um **identificador** é um conjunto de caracteres que podem ser letras, números ou *underscores* (`_`).
- O identificador deve sempre iniciar com uma letra ou o *underscore* (`_`).
- A linguagem C é **case-sensitive**, ou seja, uma palavra escrita utilizando caracteres maiúsculos é diferente da mesma palavra escrita com caracteres minúsculos.
- Palavras reservadas não podem ser usadas como nome de variáveis.
- As **palavras reservadas** formam a sintaxe da linguagem e possuem funções específicas.

Palavras reservadas da linguagem C

A partir do padrão C11 existem 44 palavras reservadas

auto	break	case	char
const	continue	default	do
double	else	enum	extern
float	for	goto	if
inline	int	long	register
restrict	return	short	signed
sizeof	static	struct	switch
typedef	union	unsigned	void
volatile	while	_Alignas	_Alignof
_Atomic	_Bool	_Complex	_Generic
_Imaginary	_Noreturn	_Static_assert	_Thread_local

Tipos nativos da linguagem C (64 bits)

Tipo	Bytes	Faixa de valores
char / signed char	1	-128 ... 127
unsigned char	1	0 ... 255
short	2	-32.768 ... 32.767
unsigned short	2	0 ... 65.535
int	4	-2.147.483.648 ... 2.147.483.647
unsigned int	4	0 ... 4.294.967.295
long	8	-9.223e18 ... 9.223e18
unsigned long	8	0 ... 1.844e19
long long	8	-9.223e18 ... 9.223e18
unsigned long long	8	0 ... 1.844e19
float	4	$\pm 1.175\text{e-}38$... $\pm 3.403\text{e}38$
double	8	$\pm 2.225\text{e-}308$... $\pm 1.798\text{e}308$
long double	16	depende da implementação

Operador sizeof

Podemos exibir o número de bytes requeridos por um tipo na nossa máquina usando o operador `sizeof()`:

```
1 #include <stdio.h> // prog05.c
2
3 int main() {
4     printf("char: %ld\n", sizeof(char));
5     printf("short: %ld\n", sizeof(short));
6     printf("int: %ld\n", sizeof(int));
7     printf("long: %ld\n", sizeof(long int));
8     printf("float: %ld\n", sizeof(float));
9     printf("double: %ld\n", sizeof(double));
10    printf("long double: %ld\n", sizeof(long double));
11 }
```

Conversão de tipos em C

- **Conversão de tipo** é o processo de conversão de um valor de um tipo de dados para outro tipo.
- Há dois modos de se converter tipos em C:

Conversão de tipos em C

- **Conversão de tipo** é o processo de conversão de um valor de um tipo de dados para outro tipo.
- Há dois modos de se converter tipos em C:
 - **Conversão Implícita ou Coerção**: em que o compilador transforma automaticamente um tipo de dados fundamental em outro.
 - **Conversão Explícita (Casting)**: em que o programador usa um operador de *casting* para direcionar a conversão.

Conversão de tipos Implícita

Existem dois tipos de conversão implícita:

- **Promoção Numérica:** Quando um valor de um tipo é implicitamente convertido para um tipo semelhante **maior**.

char → short → int → long → float → double

Promoções **não resultam** em perda de dados.

Conversão de tipos Implícita

Existem dois tipos de conversão implícita:

- **Promoção Numérica:** Quando um valor de um tipo é implicitamente convertido para um tipo semelhante **maior**.

$\text{char} \rightarrow \text{short} \rightarrow \text{int} \rightarrow \text{long} \rightarrow \text{float} \rightarrow \text{double}$

Promoções **não resultam** em perda de dados.

- **Rebaixamento Numérico:** Quando um valor de um tipo é implicitamente convertido para um tipo semelhante **menor**.

$\text{char} \leftarrow \text{short} \leftarrow \text{int} \leftarrow \text{long} \leftarrow \text{float} \leftarrow \text{double}$

Rebaixamentos numéricos **resultam** em perda de dados.

Conversão Implícita — Exemplo

```
1 #include <stdio.h> // prog29.c
2
3 int main() {
4     // Promocao numerica
5     int v = 30;
6     float f = v;
7     printf("%f\n", f/7); // Imprime o valor 4.28571
8     printf("%d\n", v/7); // Imprime o valor 4
```

Conversão Implícita — Exemplo

```
1 #include <stdio.h> // prog29.c
2
3 int main() {
4     // Promocao numerica
5     int v = 30;
6     float f = v;
7     printf("%f\n", f/7); // Imprime o valor 4.28571
8     printf("%d\n", v/7); // Imprime o valor 4
9
10    // Rebaixamento Numerico
11    int a = 35.0/4.0;
12
13    printf("%d\n", a); // Imprime o valor 8
14
15    return 0;
16 }
```


Conversão de tipos Explícita (casting)

- No exemplo abaixo, gostaríamos que a variável `f` recebesse o valor 2.5. No entanto, ela recebe o valor 2.

```
1 int i1 = 10;  
2 int i2 = 4;  
3 float f = i1 / i2;
```

Conversão de tipos Explícita (casting)

- No exemplo abaixo, gostaríamos que a variável `f` recebesse o valor 2.5. No entanto, ela recebe o valor 2.

```
1 int i1 = 10;  
2 int i2 = 4;  
3 float f = i1 / i2;
```

- Para avisarmos o compilador que queremos usar a divisão de ponto flutuante em vez da divisão de números inteiros, usamos um operador de conversão de tipos (cast).

Conversão de tipos Explícita (casting)

- No exemplo abaixo, gostaríamos que a variável `f` recebesse o valor 2.5. No entanto, ela recebe o valor 2.

```
1 int i1 = 10;  
2 int i2 = 4;  
3 float f = i1 / i2;
```

- Para avisarmos o compilador que queremos usar a divisão de ponto flutuante em vez da divisão de números inteiros, usamos um operador de conversão de tipos (cast).

Exemplo de casting:

Conversão de tipos Explícita (casting)

- No exemplo abaixo, gostaríamos que a variável `f` recebesse o valor 2.5. No entanto, ela recebe o valor 2.

```
1 int i1 = 10;  
2 int i2 = 4;  
3 float f = i1 / i2;
```

- Para avisarmos o compilador que queremos usar a divisão de ponto flutuante em vez da divisão de números inteiros, usamos um operador de conversão de tipos (cast).

Exemplo de casting:

```
1 int i1 = 10;  
2 int i2 = 4;  
3 float f = (float) i1 / i2;
```

Constantes

Uma **constante** é uma variável especial que permite guardar determinado dado na memória do computador, com a certeza de que ele não se alterará durante a execução do programa.

Constantes

Uma **constante** é uma variável especial que permite guardar determinado dado na memória do computador, com a certeza de que ele não se alterará durante a execução do programa.

A fim de declarar uma constante, basta colocar a palavra-chave **const** antes do tipo de variável:

Constantes

Uma **constante** é uma variável especial que permite guardar determinado dado na memória do computador, com a certeza de que ele não se alterará durante a execução do programa.

A fim de declarar uma constante, basta colocar a palavra-chave **const** antes do tipo de variável:

```
1 #include <stdio.h>
2
3 int main() {
4     const double pi = 3.14159;
5     // pi = 3.14; // Erro de compilacao
6     printf("pi = %f\n", pi);
7     return 0;
8 }
```

Constantes — Exemplo 2

```
1 #include <stdio.h> // prog04a.c
2
3 int main() {
4     int n;
5
6     printf("Digite o tamanho do array: ");
7     scanf("%d", &n);
8
9     const int MAX = n; // definida em tempo de execucao
10
11     int array[MAX];
12
13     for(int i = 0; i < MAX; i++) {
14         printf("array[%d] = %d\n", i, array[i]);
15     }
16
17     return 0;
18 }
```


Constantes

O C suporta dois tipos de constantes:

- **Constantes em tempo de compilação:** são aquelas cujos valores de inicialização podem ser resolvidos em tempo de compilação.
Exemplo: variável **pi** do exemplo anterior.

Constantes

O C suporta dois tipos de constantes:

- **Constantes em tempo de compilação:** são aquelas cujos valores de inicialização podem ser resolvidos em tempo de compilação.
Exemplo: variável **pi** do exemplo anterior.
 - Constantes deste tipo permitem que o compilador realize otimizações.

Constantes

O C suporta dois tipos de constantes:

- **Constantes em tempo de compilação:** são aquelas cujos valores de inicialização podem ser resolvidos em tempo de compilação.
Exemplo: variável **pi** do exemplo anterior.
 - Constantes deste tipo permitem que o compilador realize otimizações.
- **Constantes em tempo de execução:** são aquelas cujos valores de inicialização só podem ser resolvidos em tempo de execução.
Exemplo: variável **MAX**, do exemplo anterior.

Escrevendo na tela



Função printf

- A função `printf()` é uma das funções de escrita de dados de C.

Função printf

- A função `printf()` é uma das funções de escrita de dados de C.
- Seu nome vem da expressão em inglês `print formatted`, ou seja, escrita formatada.

Função printf

- A função `printf()` é uma das funções de escrita de dados de C.
- Seu nome vem da expressão em inglês `print formatted`, ou seja, escrita formatada.
- Basicamente, a função `printf()` escreve na saída de vídeo (tela) um conjunto de valores, caracteres e/ou sequência de caracteres de acordo com o formato especificado.

Função printf

- A função `printf()` é uma das funções de escrita de dados de C.
- Seu nome vem da expressão em inglês `print formatted`, ou seja, escrita formatada.
- Basicamente, a função `printf()` escreve na saída de vídeo (tela) um conjunto de valores, caracteres e/ou sequência de caracteres de acordo com o formato especificado.
- A forma geral da função `printf()` é:

```
printf(“tipos de saída”, lista de variáveis);
```


Exemplo: Escrevendo uma mensagem de texto

```
1 #include <stdio.h> // prog100.c
2
3 int main() {
4     printf("Esse texto sera escrito na tela.\n");
5     return 0;
6 }
```

Escrevendo valores formatados

- Quando queremos escrever dados formatados na tela, usamos a forma geral da função, a qual possui os **tipos de saída** ou **especificador de formato**.

Escrevendo valores formatados

- Quando queremos escrever dados formatados na tela, usamos a forma geral da função, a qual possui os **tipos de saída** ou **especificador de formato**.
- Cada tipo de saída é precedido por um sinal de %, e um tipo de saída deve ser especificado para cada variável a ser escrita.

Escrevendo valores formatados

- Quando queremos escrever dados formatados na tela, usamos a forma geral da função, a qual possui os **tipos de saída** ou **especificador de formato**.
- Cada tipo de saída é precedido por um sinal de %, e um tipo de saída deve ser especificado para cada variável a ser escrita.
- se quiséssemos escrever uma única expressão com o comando `printf()`, faríamos:

```
printf(“%tipo_de_saida”, expressao);
```

Escrevendo valores formatados

- Quando queremos escrever dados formatados na tela, usamos a forma geral da função, a qual possui os **tipos de saída** ou **especificador de formato**.
- Cada tipo de saída é precedido por um sinal de %, e um tipo de saída deve ser especificado para cada variável a ser escrita.
- se quiséssemos escrever uma única expressão com o comando `printf()`, faríamos:

```
printf('%tipo_de_saida', expressao);
```

- Se fossem duas as expressões a serem escritas, faríamos:

```
printf('%tipo1', '%tipo2', expressao1, expressao2);
```

Especificadores de Formato do printf()

Especificador	Tipo de dado impresso
%d, %i	int (decimal com sinal)
%u	unsigned int (decimal sem sinal)
%o	unsigned int (octal)
%x, %X	unsigned int (hexadecimal, min/maiúsculo)
%f	double ou float (ponto flutuante decimal)
%F	double ou float (ponto flutuante decimal, maiúsculo)
%c	int (caractere único, convertido de unsigned char)
%s	char* (string terminada em \0)
%p	void* (endereço de ponteiro)
%%	imprime o caractere %

Exemplo 1: Escrevendo valores na tela

```
1 #include <stdio.h> // prog101.c
2
3 int main() {
4     printf("%d\n", 55);           // int
5     printf("%i\n", 234);         // int
6     printf("%f\n", 55.56f);      // float
7     printf("%f\n", 77.654321);   // double
8     printf("%F\n", 77.654321);   // double
9     printf("%c\n", 67);          // char
10    printf("%s\n", "ana maria"); // char*
11    printf("%%\n");
12    return 0;
13 }
```

Modificadores de Comprimento do printf()

Modificador	Tipo de dado correspondente
h	short int ou unsigned short int
l	long int, unsigned long int
ll	long long int ou unsigned long long int
z	size_t
t	ptrdiff_t
L	long double

- Os modificadores de comprimento que podem ser usados junto com os especificadores de formato do printf() no C.

Exemplo 2: Escrevendo valores na tela

```
1 #include <stdio.h> // prog102.c
2
3 int main() {
4     short v = 87;
5     long z = 2356432;
6     size_t x = 7654;
7     long long w = 87654321;
8     long double ldo = 771.65432132189;
9     printf("%hd\n", v);      // short int
10    printf("%ld\n", z);      // long int
11    printf("%zu\n", x);      // size_t
12    printf("%lld\n", w);     // long long int
13    printf("%Lf\n", ldo);    // long double
14    return 0;
15 }
```

Caracteres de Escape em C

Sequência	Significado
<code>\n</code>	Nova linha (line feed)
<code>\t</code>	Tabulação horizontal
<code>\v</code>	Tabulação vertical
<code>\b</code>	Backspace
<code>\r</code>	Retorno de carro (carriage return)
<code>\f</code>	Form feed (avanço de página)
<code>\a</code>	Alerta sonoro (bell)
<code>\0</code>	Caractere nulo (terminador de string)
<code>\'</code>	Aspas simples
<code>\"</code>	Aspas duplas
<code>\?</code>	Ponto de interrogação literal
<code>\\</code>	Barra invertida

Exemplo 3: caracteres de escape

```
1 #include <stdio.h> // prog103.c
2
3 int main() {
4     printf("\?\n");           // ?
5     printf("\'atilio\'\n");    // 'atilio'
6     printf("\"ana\"\n");        // "ana"
7     printf("\\lucas\n");       // \lucas
8     printf("an\0a maria");     // an
9     printf("\n");
10    printf("oi,\tcomo vai?\n");
11    printf("oi\vcomo\vvai\você?\n");
12    printf("claro\bisse\n");
13    return 0;
14 }
```

Width e Alinhamento no printf()

Tipo	Comando	Saída
Inteiro	%5d	Alinhado à direita, campo mínimo 5
Inteiro	%-5d	Alinhado à esquerda, campo mínimo 5
Inteiro	.*d	Width dinâmica (p.ex. 8):
Float	%10.2f	Campo 10, 2 casas decimais, direita
Float	%-10.2f	Campo 10, 2 casas decimais, esquerda
String	%5s	Alinhado à direita, campo 5
String	%-5s	Alinhado à esquerda, campo 5
String	%.*s	Width dinâmica

Exemplo de largura e alinhamento

```
1 #include <stdio.h> // prog110.c
2
3 int main() {
4     int x = 87;
5
6     printf("|%5d|\n", x);
7     printf("|%-5d|\n", x);
8     printf("|%*d|\n", 10, x);
9     printf("|%10.2f|\n", 3.14);
10    printf("|%-10.2f|\n", 3.14);
11    printf("|%5s|\n", "Oi");
12    printf("|%-5s|\n", "Oi");
13    printf("|%-*s|\n", 8, "Oi");
14 }
```

A função `putchar()`

- A função `putchar()` (**put character**) permite escrever um único caractere na tela. Sua forma geral é:

```
int putchar(int caractere);
```

A função `putchar()`

- A função `putchar()` (**put character**) permite escrever um único caractere na tela. Sua forma geral é:

```
int putchar(int caractere);
```

- Essa função recebe como parâmetro de entrada um único valor inteiro. Esse valor será convertido em caractere e mostrado na tela.

A função `putchar()`

- A função `putchar()` (**put character**) permite escrever um único caractere na tela. Sua forma geral é:

```
int putchar(int caractere);
```

- Essa função recebe como parâmetro de entrada um único valor inteiro. Esse valor será convertido em caractere e mostrado na tela.
- Se NÃO ocorrer erro, a função retorna o próprio caractere que foi escrito. Caso contrário, a constante **EOF** (definida na biblioteca `stdio.h`) é retornada.

Exemplo: putchar()

```
1 #include <stdio.h> // prog104.c
2
3 int main() {
4     char c = 'a';
5     int x = 65;
6
7     putchar(c);           // escreve 'a'
8     putchar('\n');        // quebra de linha
9     putchar(x);           // escreve o caractere 65
10    putchar('\n');        // quebra de linha
11
12    return 0;
13 }
```

Exemplo: putchar()

```
1 #include <stdio.h> // prog104.c
2
3 int main() {
4     char c = 'a';
5     int x = 65;
6
7     putchar(c);           // escreve 'a'
8     putchar('\n');        // quebra de linha
9     putchar(x);           // escreve o caractere 65
10    putchar('\n');        // quebra de linha
11
12    return 0;
13 }
```

- a conversão em `putchar()` é direta no momento da impressão, ou seja, o valor 65 é convertido no caractere ASCII correspondente, no caso, o caractere A.
- Além disso, o comando `putchar()` também aceita o uso de sequências de escape, como o caractere `\n`.

Lendo dados do teclado



A função `scanf()`

- A função `scanf()` é uma das funções de entrada/leitura de dados da linguagem C. Seu nome vem da expressão em inglês **scan formatted**.

A função `scanf()`

- A função `scanf()` é uma das funções de entrada/leitura de dados da linguagem C. Seu nome vem da expressão em inglês **scan formatted**.
- A função `scanf()` lê do teclado um conjunto de valores, caracteres e/ou sequência de caracteres de acordo com o formato especificado. A forma geral da função é:

```
scanf(“tipos de entrada”, lista de variáveis);
```

A função `scanf()`

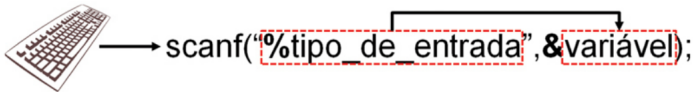
- A função `scanf()` é uma das funções de entrada/leitura de dados da linguagem C. Seu nome vem da expressão em inglês **scan formatted**.
- A função `scanf()` lê do teclado um conjunto de valores, caracteres e/ou sequência de caracteres de acordo com o formato especificado. A forma geral da função é:

```
scanf(“tipos de entrada”, lista de variáveis);
```

- Os **tipos de entrada** especificam o formato de entrada dos dados que serão lidos pela função `scanf()`. Cada tipo de entrada é precedido por um sinal de %, e um tipo de entrada deve ser especificado para cada variável a ser lida.

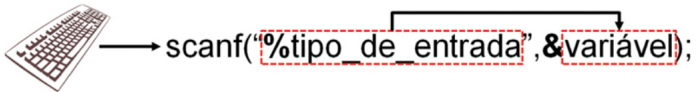
A função scanf()

- se quiséssemos ler uma única variável com o comando `scanf()`, faríamos:

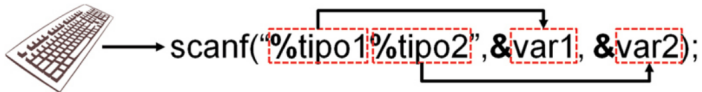


A função scanf()

- se quiséssemos ler uma única variável com o comando `scanf()`, faríamos:



- Se fossem duas as variáveis a serem lidas, faríamos:



Exemplo 1: scanf()

```
1 #include <stdio.h> // prog105.c
2
3 int main() {
4     int x, z;
5     float y;
6     double k;
7
8     // Leitura de dois inteiros
9     scanf("%d%d", &x, &z);
10
11    // Leitura de um float
12    scanf("%f", &y);
13
14    // Leitura de um double
15    scanf("%lf", &k);
16
17    printf("x=%d, z=%d, y=%f, k=%f\n", x, z, y, k);
18
19    return 0;
20 }
```

Exemplo 2: scanf()

```
1 #include <stdio.h> // prog106.c
2
3 int main() {
4     int dia, mes, ano;
5
6     printf("Digite a data de hoje ");
7     printf("no formato dd/mm/aaaa: ");
8
9     scanf("%d/%d/%d", &dia, &mes, &ano);
10
11    printf("data digitada: %d/%d/%d\n", dia,mes,ano);
12
13    return 0;
14 }
```

A função `getchar()`

- A função `getchar()` (**get character**) permite ler um único caractere do teclado. Sua forma geral é:

```
int getchar(void);
```

A função `getchar()`

- A função `getchar()` (**get character**) permite ler um único caractere do teclado. Sua forma geral é:

```
int getchar(void);
```

- Se NÃO ocorrer erro a função `getchar()` o código ASCII do caractere lido. Se ocorrer erro: a constante EOF (definida na biblioteca `stdio.h`) é retornada.

Exemplo 1: getchar()

```
1 #include <stdio.h> // prog107.c
2
3 int main() {
4     char nome[35];
5     printf("Digite o seu primeiro nome: ");
6
7     scanf("%s", nome);
8
9     getchar(); // lê o \n que fica no buffer
10
11    printf("seu nome é: %s\n", nome);
12
13    printf("digite algo para encerrar...");
14
15    getchar();
16
17    return 0;
18 }
```

Exemplo 2: getchar()

```
1 #include <stdio.h> // prog108.c
2
3 int main() {
4     int idade;
5     printf("Digite a sua idade: ");
6
7     scanf("%d", &idade);
8
9     getchar(); // lê o \n que fica no buffer
10
11    printf("sua idade é: %d\n", idade);
12
13    printf("digite algo para encerrar...");
14
15    getchar();
16
17    return 0;
18 }
```

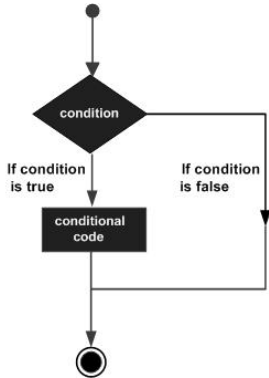
Exemplo 3: getchar()

```
1 #include <stdio.h> // prog108.c
2
3 int main() {
4     int idade;
5     printf("Digite a sua idade: ");
6     scanf("%d", &idade);
7
8     char nome[35];
9     printf("Digite o seu primeiro nome: ");
10    scanf("%s", nome);
11
12    printf("sua idade é: %d\n", idade);
13    printf("seu nome é: %s\n", nome);
14    printf("digite algo para encerrar...");
15
16    getchar(); // lê o \n que fica no buffer após scanf()
17    getchar();
18
19    return 0;
20 }
```

Estruturas de Seleção

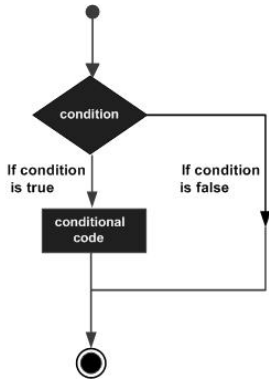


Estruturas de Seleção — If.. else



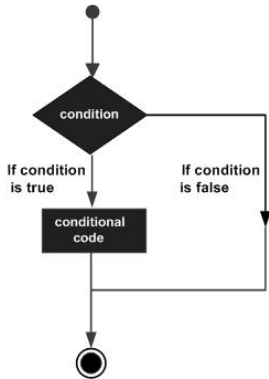
```
1 int c;  
2 scanf("%d", &c);  
3  
4 if (c == 1) { // If  
5     printf("igual a 1");  
6 }
```

Estruturas de Seleção — If.. else



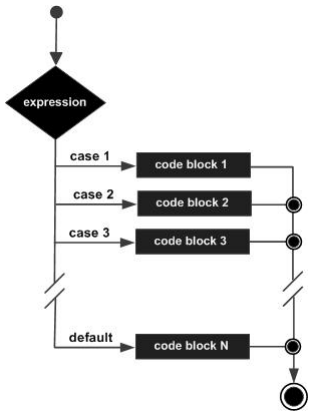
```
1 int c;
2 scanf("%d", &c);
3
4 if (c == 1) { // If
5     printf("igual a 1");
6 }
7
8 if (c == 2) { // If .. else
9     printf("igual a 2");
10 } else {
11     printf("nao eh 1 nem 2");
12 }
```

Estruturas de Seleção — If.. else



```
1 int c;
2 scanf("%d", &c);
3
4 if (c == 1) { // If
5     printf("igual a 1");
6 }
7
8 if (c == 2) { // If .. else
9     printf("igual a 2");
10 } else {
11     printf("nao eh 1 nem 2");
12 }
13
14 if (c == 3) { // If else encadeado
15     printf("igual a 3");
16 } else if (c == 4) {
17     printf("igual a 4");
18 } else {
19     printf("nao eh 3 nem 4");
20 }
```

Estruturas de Seleção — Switch



```
1 int c;  
2 scanf("%d", &c);  
3  
4 switch (c) // int, char, short, long  
5 {  
6     case 1:  
7         printf("1 \n");  
8         break;  
9     case 2:  
10        printf("2 \n");  
11        break;  
12    case 3:  
13        printf("3 \n");  
14        break;  
15    case 4:  
16        printf("4 \n");  
17        break;  
18    default:  
19        printf("5 \n");  
20        break;  
21 }
```

Alternativa para o if-else

Operador ternário

O **operador ternário** avalia uma condição de teste e executa um bloco de código com base no resultado da condição.

Sua sintaxe é:

```
condition ? expression1 : expression2 ;
```

- se **condition** for **true** então **expression1** é executado;
- se **condition** for **false** então **expression2** é executado.

Operador ternário – Exemplo

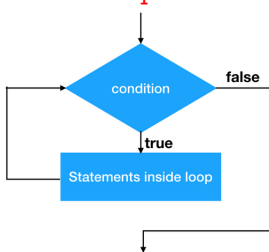
```
1 #include <stdio.h> // prog150.c
2
3 int main() {
4     double nota;
5
6     printf("Digite sua nota: ");
7     scanf("%lf", &nota);
8
9     printf("Voce %s no exame.\n",
10         nota >= 7 ? "passou" : "reprovou");
11
12     return 0;
13 }
```

Estruturas de Repetição



Estruturas de repetição (loops)

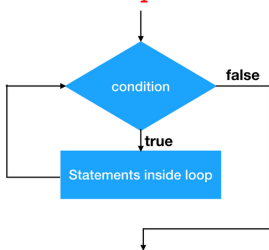
while loop



```
1 int contador = 0;
2
3 while (contador < 10) {
4     printf("%d ", contador);
5     contador++;
6 }
```

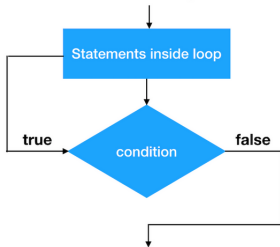

Estruturas de repetição (loops)

while loop



```
1 int contador = 0;
2
3 while (contador < 10) {
4     printf("%d ", contador);
5     contador++;
6 }
```

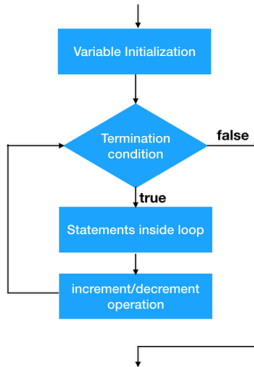
do..while loop



```
1 int contador = 0;
2
3 do {
4     printf("%d ", contador);
5     contador++;
6 } while (contador < 10);
```

Estruturas de repetição (loops)

for loop



```
1 for (int i = 0; i < 10; i++) {  
2     printf("%d ", i);  
3 }
```

Exercícios

- Faça um programa que mostre ao usuário um menu com quatro opções de operações matemáticas (as operações básicas, por exemplo). O usuário escolhe uma das opções, e o seu programa pede dois valores numéricos e realiza a operação, mostrando o resultado.
- Usando o comando **switch**, escreva um programa que leia um inteiro entre 1 e 7 e imprima o dia da semana correspondente a esse número. Isto é, domingo, se 1, segunda-feira, se 2, e assim por diante.
- Faça um programa que leia os coeficientes de uma equação do segundo grau. Em seguida, calcule e mostre as raízes dessa equação.

- Escreva um programa que leia um número inteiro N , maior ou igual a zero, fornecido pelo usuário e imprima o N -ésimo termo da sequência de Fibonacci.

A sequência de Fibonacci começa no termo de ordem zero, e, a partir do segundo termo, seu valor é dado pela soma dos dois termos anteriores. Alguns termos dessa sequência são: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34.

- Faça um programa que leia um valor inteiro e positivo N , calcule e mostre o valor da constante de Euler E . Essa constante é definida conforme a fórmula a seguir:

$$E = \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \cdots + \frac{1}{N!}$$

Exercícios

- Escreva um programa que leia certa quantidade de números, imprima o maior deles e quantas vezes o maior número foi lido. A quantidade de números a serem lidos deve ser fornecida pelo usuário.

Vetores

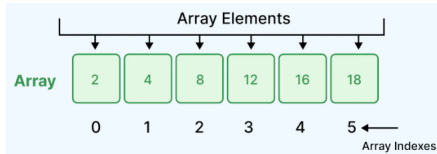


Vetores (Arrays)

- O C também suporta **variáveis agregadas**, que podem armazenar coleções de valores. Existem dois tipos de variáveis agregadas:
 - **arrays**: dados homogêneos
 - **structs**: permite dados heterogêneos

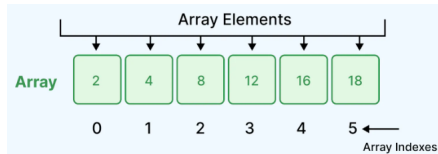
Vetores (Arrays)

- O C também suporta **variáveis agregadas**, que podem armazenar coleções de valores. Existem dois tipos de variáveis agregadas:
 - **arrays**: dados homogêneos
 - **structs**: permite dados heterogêneos
- Um **vetor (array)** é uma estrutura de dados que nos permite acessar muitas variáveis do mesmo tipo por meio de um único identificador.



Vetores (Arrays)

- O C também suporta **variáveis agregadas**, que podem armazenar coleções de valores. Existem dois tipos de variáveis agregadas:
 - **arrays**: dados homogêneos
 - **structs**: permite dados heterogêneos
- Um **vetor (array)** é uma estrutura de dados que nos permite acessar muitas variáveis do mesmo tipo por meio de um único identificador.



- Em quais casos na programação pode ser necessário o uso de um vetor?

Declarando e inicializando vetores

<code>int numbers[10];</code>	Um array com capacidade para 10 inteiros não inicializado.
<code>const int SIZE = 10; int numbers[SIZE];</code>	É uma boa ideia usar uma variável constante para o tamanho.
<code>int size = 10; int numbers[size];</code>	Atenção: Variable Length Arrays (VLA) desde C99.
<code>int vec[5] = {0,1,4,9,16};</code>	Um vetor de cinco inteiros, inicializado com "brace initialization"
<code>int vec[] = {0,1,4,9,16};</code>	O tamanho do vetor pode ser omitido neste caso, pois ele é definido pelo número de valores iniciais.
<code>int squares[5] = {0,1,4};</code>	Se você fornecer menos valores iniciais que o tamanho, os valores restantes serão definidos como 0. Esse array contém 0, 1, 4, 0, 0.
<code>int squares[] = {[8] = 13};</code>	Inicializa somente os índices fornecidos.

Tamanho de um vetor

Se estivermos no mesmo escopo em que um array foi definido, podemos determinar seu tamanho usando o operador `sizeof`.

Exemplo:

Tamanho de um vetor

Se estivermos no mesmo escopo em que um array foi definido, podemos determinar seu tamanho usando o operador `sizeof`.

Exemplo:

```
1 #include <stdio.h> // prog16.c
2
3 int main() {
4     int array[] = { 1, 1, 2, 3, 5, 8, 13, 21 };
5
6     size_t n = sizeof(array) / sizeof(array[0]);
7
8     printf("Tamanho do array: %zu\n", n);
9
10    return 0;
11 }
```

- **Obs.:** Só funcionará se `sizeof` estiver na mesma função/escopo no qual o vetor estiver declarado.

Exemplo de array

```
1 #include <stdio.h> // prog17.c
2
3 int main() {
4     int array[] = { 0, [1] = 1, [5] = 2, [8] = 13 };
5
6     int n = sizeof(array) / sizeof(array[0]);
7
8     for(int i = 0; i < n; ++i) {
9         printf("[%d] = %d\n", i, array[i]);
10    }
11
12    return 0;
13 }
```

Arrays multidimensionais (matrizes)

- Um array pode ter qualquer número de dimensões.
- A seguinte declaração cria uma matriz:

```
int mat[5][9];
```

Arrays multidimensionais (matrizes)

- Um array pode ter qualquer número de dimensões.
- A seguinte declaração cria uma matriz:

```
int mat[5][9];
```

A matriz tem capacidade para 5 linhas e 9 colunas.

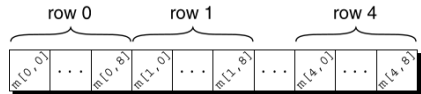
	0	1	2	3	4	5	6	7	8
0									
1									
2									
3									
4									

A fim de acessar o elemento na linha i e coluna j da matriz mat , devemos escrever $mat[i][j]$.

Representação interna de matrizes

- Embora visualizemos uma matriz como uma tabela, esse não é o jeito que os elementos estão de fato representados na memória.

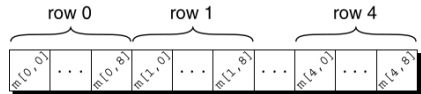
	0	1	2	3	4	5	6	7	8
0									
1									
2									
3									
4									



Representação interna de matrizes

- Embora visualizemos uma matriz como uma tabela, esse não é o jeito que os elementos estão de fato representados na memória.
- C armazena matriz em uma ordem chamada **row-major order**, com a linha 0 primeiro, depois linha 1, e assim por diante.

	0	1	2	3	4	5	6	7	8
0									
1									
2									
3									
4									



Exemplo: Criando matriz identidade

```
1 #include <stdio.h> // prog120.c
2
3 int main() {
4     const int N = 5;
5     int mat[N][N];
6
7     for(int row = 0; row < N; ++row) {
8         for(int col= 0; col < N; ++col) {
9             if(row == col)
10                 mat[row][col] = 1;
11             else
12                 mat[row][col] = 0;
13         }
14     }
15
16     for(int row = 0; row < N; ++row) {
17         for(int col= 0; col < N; ++col) {
18             printf("%d ", mat[row][col]);
19         }
20         printf("\n");
21     }
22     return 0;
23 }
```

Exercícios

- Faça um programa para ler 10 números diferentes a serem armazenados em um vetor. Os números deverão ser armazenados no vetor na ordem em que forem lidos, sendo que, caso o usuário digite um número que já foi digitado, o programa deverá pedir a ele para digitar outro número. Note que cada valor digitado pelo usuário deve ser pesquisado no vetor, verificando se ele existe entre os números que já foram fornecidos. Exiba na tela o vetor final que foi digitado.

- Faça um programa que calcule o desvio-padrão d de um vetor V contendo n números. O valor de d é dado pela fórmula:

$$d = \sqrt{\frac{1}{n-1} \sum_{i=0}^{n-1} (V[i])^2 - m^2}$$

em que m é a média desse vetor. Considere $n = 10$. O vetor V deve ser lido do teclado.

Exercícios

- Leia um vetor com 15 números de ponto flutuante. Em seguida, ordene os elementos desse vetor e imprima o vetor na tela.

Exercícios

Leia uma matriz de números de tamanho 4×4 e faça o seguinte:

- Em seguida, imprima a soma dos valores contidos em sua diagonal principal.
- Em seguida, imprima a soma dos valores contidos em sua diagonal secundária.
- Em seguida, conte e escreva na tela quantos valores negativos ela possui.
- Calcule e imprima a soma dos elementos dessa matriz que estão acima da diagonal principal.

Exercícios

- Faça um programa que leia uma matriz A de tamanho 5×5 . Em seguida, calcule e imprima a matriz B , sendo que $B = A^2$.

Arrays de caracteres – Strings do C



Arrays de caracteres — Strings do C

- Na linguagem C, uma **string** é uma sequência de caracteres terminados pelo caractere `'\0'`.

Arrays de caracteres — Strings do C

- Na linguagem C, uma **string** é uma sequência de caracteres terminados pelo caractere `'\0'`.
 - É armazenada na memória do computador na forma de um array do tipo `char`. Exemplo de declaração + inicialização:

```
char palavra[6] = 'oi';
```

o	i	\0	?	ã	#
---	---	----	---	---	---

Arrays de caracteres — Strings do C

- Na linguagem C, uma **string** é uma sequência de caracteres terminados pelo caractere `'\0'`.
 - É armazenada na memória do computador na forma de um array do tipo `char`. Exemplo de declaração + inicialização:

```
char palavra[6] = 'oi';
```

o	i	\0	?	ã	#
---	---	----	---	---	---

- Nesse caso, temos 3 posições não utilizadas e que estão preenchidas com lixo de memória (um valor qualquer).

Arrays de caracteres — Strings do C

- Na linguagem C, uma **string** é uma sequência de caracteres terminados pelo caractere `'\0'`.
 - É armazenada na memória do computador na forma de um array do tipo `char`. Exemplo de declaração + inicialização:

```
char palavra[6] = 'oi';
```

o	i	\0	?	ã	#
---	---	----	---	---	---

- Nesse caso, temos 3 posições não utilizadas e que estão preenchidas com lixo de memória (um valor qualquer).
- Como o caractere `\0` indica o final de nossa string, isso significa que, em uma string definida com tamanho de 6 caracteres, apenas 5 estarão disponíveis para armazenar o texto digitado pelo usuário.

Strings do C — Inicialização

- **Inicialização de strings:** strings seguem as mesmas regras que os arrays: você pode inicializar a string no momento da criação, mas você não pode atribuir valores a ela usando o operador de atribuição depois disso.

Strings do C — Inicialização

- **Inicialização de strings:** strings seguem as mesmas regras que os arrays: você pode inicializar a string no momento da criação, mas você não pode atribuir valores a ela usando o operador de atribuição depois disso.
- Primeira forma de inicialização:

```
char palavra[6] = "oi";
```

Strings do C — Inicialização

- **Inicialização de strings:** strings seguem as mesmas regras que os arrays: você pode inicializar a string no momento da criação, mas você não pode atribuir valores a ela usando o operador de atribuição depois disso.

- Primeira forma de inicialização:

```
char palavra[6] = "oi";
```

- Segunda forma de inicialização:

```
char palavra[6] = { 'o', 'i', '\0' };
```


Strings do C — Modificando uma string

- A linguagem C não suporta a atribuição de um array para outro. Logo, não é possível usar o operador de atribuição entre strings.

Strings do C — Modificando uma string

- A linguagem C não suporta a atribuição de um array para outro. Logo, não é possível usar o operador de atribuição entre strings.

```
1 #include <stdio.h> // prog121.c
2
3 int main() {
4     char palavra1[6] = "oi";
5     char palavra2[6];
6
7     palavra2 = palavra1; // ERRADO!!!
8
9     return 0;
10 }
```

Strings do C — Modificando uma string

- A linguagem C não suporta a atribuição de um array para outro. Logo, não é possível usar o operador de atribuição entre strings.

```
1 #include <stdio.h> // prog121.c
2
3 int main() {
4     char palavra1[6] = "oi";
5     char palavra2[6];
6
7     palavra2 = palavra1; // ERRADO!!!
8
9     return 0;
10 }
```

- Para atribuir o conteúdo de uma string a outra, o correto é copiar a string, elemento por elemento, para a outra string.

Exemplo: copiando uma string

```
1  #include <stdio.h>
2
3  int main() {
4      char str1[6] = "solar";
5      char str2[6];
6      int i = 0;
7
8      for(i = 0; str1[i] != '\0'; ++i) {
9          str2[i] = str1[i];
10     }
11     str2[i] = '\0';
12
13     printf("%s\n", str2);
14     return 0;
15 }
```

Observação

- Infelizmente, o tipo de manipulação de arrays mostrado no código do slide anterior não é muito prático quando estamos trabalhando com palavras.

- Infelizmente, o tipo de manipulação de arrays mostrado no código do slide anterior não é muito prático quando estamos trabalhando com palavras.
- Felizmente, a biblioteca-padrão da linguagem C possui funções especialmente desenvolvidas para a manipulação de strings na biblioteca `<string.h>`

Lendo strings do teclado — `fgets()`

- A função `fgets()` faz a leitura do teclado considerando todos os caracteres digitados (incluindo os espaços) até encontrar uma tecla enter:

```
char *fgets(char *str, int tamanho, FILE *fp);
```

Lendo strings do teclado — `fgets()`

- A função `fgets()` faz a leitura do teclado considerando todos os caracteres digitados (incluindo os espaços) até encontrar uma tecla enter:

```
char *fgets(char *str, int tamanho, FILE *fp);
```

- A função `fgets()` recebe três parâmetros de entrada:

Lendo strings do teclado — `fgets()`

- A função `fgets()` faz a leitura do teclado considerando todos os caracteres digitados (incluindo os espaços) até encontrar uma tecla enter:

```
char *fgets(char *str, int tamanho, FILE *fp);
```

- A função `fgets()` recebe três parâmetros de entrada:
 - **str**: a string a ser lida.

Lendo strings do teclado — `fgets()`

- A função `fgets()` faz a leitura do teclado considerando todos os caracteres digitados (incluindo os espaços) até encontrar uma tecla enter:

```
char *fgets(char *str, int tamanho, FILE *fp);
```

- A função `fgets()` recebe três parâmetros de entrada:
 - **str**: a string a ser lida.
 - **tamanho**: o limite máximo de caracteres a serem lidos, incluindo o `'\0'`.

Lendo strings do teclado — `fgets()`

- A função `fgets()` faz a leitura do teclado considerando todos os caracteres digitados (incluindo os espaços) até encontrar uma tecla `enter`:

```
char *fgets(char *str, int tamanho, FILE *fp);
```

- A função `fgets()` recebe três parâmetros de entrada:
 - **str**: a string a ser lida.
 - **tamanho**: o limite máximo de caracteres a serem lidos, incluindo o `'\0'`.
 - **fp**: a variável que está associada ao arquivo de onde a string será lida.

Lendo strings do teclado — `fgets()`

- A função `fgets()` faz a leitura do teclado considerando todos os caracteres digitados (incluindo os espaços) até encontrar uma tecla enter:

```
char *fgets(char *str, int tamanho, FILE *fp);
```

- A função `fgets()` recebe três parâmetros de entrada:
 - **str**: a string a ser lida.
 - **tamanho**: o limite máximo de caracteres a serem lidos, incluindo o `'\0'`.
 - **fp**: a variável que está associada ao arquivo de onde a string será lida.
- Ela retorna um ponteiro para o primeiro caractere da string recuperada em `str`, ou `NULL` caso dê algum erro.

Lendo strings do teclado — `fgets()`

- A função `fgets()` faz a leitura do teclado considerando todos os caracteres digitados (incluindo os espaços) até encontrar uma tecla enter:

```
char *fgets(char *str, int tamanho, FILE *fp);
```

- A função `fgets()` recebe três parâmetros de entrada:
 - **str**: a string a ser lida.
 - **tamanho**: o limite máximo de caracteres a serem lidos, incluindo o `'\0'`.
 - **fp**: a variável que está associada ao arquivo de onde a string será lida.
- Ela retorna um ponteiro para o primeiro caractere da string recuperada em `str`, ou `NULL` caso dê algum erro.
- **Atenção:** Na função `fgets()`, o caractere de nova linha (`'\n'`) fará parte da string se ele couber no array. Sempre adiciona o terminador `'\0'` ao final da string lida.

Exemplo de uso — fgets()

```
1  #include <stdio.h> // prog124.c
2
3  int main(void) {
4      char buffer[20];
5
6      printf("Digite algo: ");
7      if (fgets(buffer, sizeof(buffer), stdin) != NULL) {
8          // note que pode incluir '\n'
9          printf("Você digitou: %s", buffer);
10     } else {
11         printf("Erro na leitura!\n");
12     }
13
14     return 0;
15 }
```

Limpando o buffer do teclado (`stdin`)

- Às vezes, podem ocorrer erros durante a leitura de caracteres ou strings do teclado e alguns caracteres podem sobrar no buffer de entrada `stdin`.

Limpando o buffer do teclado (`stdin`)

- Às vezes, podem ocorrer erros durante a leitura de caracteres ou strings do teclado e alguns caracteres podem sobrar no buffer de entrada `stdin`.
- Assim, devemos limpar o buffer do teclado (entrada-padrão) antes de ler novas strings.

Limpando o buffer do teclado (`stdin`)

- Às vezes, podem ocorrer erros durante a leitura de caracteres ou strings do teclado e alguns caracteres podem sobrar no buffer de entrada `stdin`.
- Assim, devemos limpar o buffer do teclado (entrada-padrão) antes de ler novas strings.
- Uma forma segura de fazer isso é utilizar a função `getchar()` para ler os caracteres restantes até encontrar um caractere de nova linha (`'\n'`) ou a constante EOF.

Limpando o buffer do teclado (`stdin`)

- Às vezes, podem ocorrer erros durante a leitura de caracteres ou strings do teclado e alguns caracteres podem sobrar no buffer de entrada `stdin`.
- Assim, devemos limpar o buffer do teclado (entrada-padrão) antes de ler novas strings.
- Uma forma segura de fazer isso é utilizar a função `getchar()` para ler os caracteres restantes até encontrar um caractere de nova linha (`'\n'`) ou a constante EOF.
- Exemplo:

Limpando o buffer do teclado (`stdin`)

- Às vezes, podem ocorrer erros durante a leitura de caracteres ou strings do teclado e alguns caracteres podem sobrar no buffer de entrada `stdin`.
- Assim, devemos limpar o buffer do teclado (entrada-padrão) antes de ler novas strings.
- Uma forma segura de fazer isso é utilizar a função `getchar()` para ler os caracteres restantes até encontrar um caractere de nova linha (`'\n'`) ou a constante EOF.
- Exemplo:

```
1 void clear_stdin() {  
2     int c;  
3     // descarta todos os caracteres até '\n' ou EOF  
4     while ((c = getchar()) != '\n' && c != EOF);  
5 }
```

Exemplo: leitura + limpeza

```
1 #include <stdio.h> // prog126.c
2 #include <string.h>
3
4 void clear_stdin(void) {
5     int c;
6     while ((c = getchar()) != '\n' && c != EOF);
7 }
```

```
8  int main() {
9      char buffer[5];          // buffer pequeno para exemplo
10     int max_iter = 3;        // número de leituras
11
12     for (int i = 0; i < max_iter; i++) {
13         printf("Digite uma palavra (max 4 chars): ");
14
15         if (fgets(buffer, 5, stdin) == NULL) {
16             return 0; // erro ou EOF
17         }
18
19         size_t len = strlen(buffer);
20
21         // Remove '\n' se estiver presente
22         if (len > 0 && buffer[len - 1] == '\n') {
23             buffer[len - 1] = '\0';
24         } else {
25             // Sobrou excesso no stdin, limpar
26             clear_stdin();
27         }
28         printf("Você digitou: %s\n", buffer);
29     }
30     return 0;
31 }
```

Escrevendo uma string na tela — `printf()`

- Basicamente, para escrever uma string na tela utilizamos a função `printf()` com o formato de dados “%s”:

Escrevendo uma string na tela — printf()

- Basicamente, para escrever uma string na tela utilizamos a função `printf()` com o formato de dados “%s”:
- Exemplo:

```
char str[20] = ‘‘Hello World’’;  
printf(‘‘%s’’,str);
```

Escrevendo uma string na tela — `fputs()`

- Outra função que também permite a escrita de strings na tela. Essa função é a `fputs()`, cujo protótipo é:

```
int fputs(char *str, FILE *fp);
```


Escrevendo uma string na tela — `fputs()`

- Outra função que também permite a escrita de strings na tela. Essa função é a `fputs()`, cujo protótipo é:

```
int fputs(char *str, FILE *fp);
```

- A função `fputs()` recebe dois parâmetros de entrada:

Escrevendo uma string na tela — `fputs()`

- Outra função que também permite a escrita de strings na tela. Essa função é a `fputs()`, cujo protótipo é:

```
int fputs(char *str, FILE *fp);
```

- A função `fputs()` recebe dois parâmetros de entrada:
 - **str**: a string (array de caracteres) a ser escrita na tela.

Escrevendo uma string na tela — `fputs()`

- Outra função que também permite a escrita de strings na tela. Essa função é a `fputs()`, cujo protótipo é:

```
int fputs(char *str, FILE *fp);
```

- A função `fputs()` recebe dois parâmetros de entrada:
 - **str**: a string (array de caracteres) a ser escrita na tela.
 - **fp**: a variável que está associada ao arquivo onde a string será escrita.

Escrevendo uma string na tela — `fputs()`

- Outra função que também permite a escrita de strings na tela. Essa função é a `fputs()`, cujo protótipo é:

```
int fputs(char *str, FILE *fp);
```

- A função `fputs()` recebe dois parâmetros de entrada:
 - **str**: a string (array de caracteres) a ser escrita na tela.
 - **fp**: a variável que está associada ao arquivo onde a string será escrita.
- `fputs()` retorna a constante EOF (em geral, -1), se houver erro na escrita, ou um valor diferente de ZERO, se o texto for escrito com sucesso.

Escrevendo uma string na tela — fputs()

```
1 #include <stdio.h>
2
3 int main() {
4     char str[15] = "paralelepipedo";
5
6     fputs(str, stdout);
7     fputc('\n', stdout); // imprime apenas um caractere
8
9     return 0;
10 }
```

Manipulando strings (biblioteca `string.h`)

Duas funções da biblioteca `string.h`:

- `int strlen(const char *str);`

Retorna o número de caracteres da string `str`, excluindo o caractere nulo de terminação.

Manipulando strings (biblioteca `string.h`)

Duas funções da biblioteca `string.h`:

- `int strlen(const char *str);`
Retorna o número de caracteres da string `str`, excluindo o caractere nulo de terminação.
- `char* strcpy(char *destino, const char *origem);`
Copia a string `origem` para a string `destino`, incluindo o caractere nulo de terminação (e parando nesse ponto) e retorna o endereço da string destino.
Para evitar overflows, o tamanho do vetor `destino` deve ser longo o suficiente para conter a string `origem`.

Exemplo com strlen e strcpy

```
1 #include <stdio.h> // prog20.c
2 #include <string.h>
3
4 int main () {
5     char str1[] = "oi gente";
6     char str2[40];
7     char str3[40];
8
9     strcpy(str2, str1);
10
11     strcpy(str3, "Copia bem sucedida");
12
13     printf("str1: %s, str2: %s\n", str1, str2);
14     printf("str3: %s\n", str3);
15     printf("tamanho de str1: %zu\n", strlen(str1));
16
17     return 0;
18 }
```


Comparando duas strings

- Da mesma maneira como o operador de atribuição não funciona para strings, o mesmo ocorre com operadores relacionais usados para comparar duas strings. Desse modo, para saber se duas strings são iguais usa-se a função `strcmp()`:

```
int strcmp(const char *str1, const char *str2);
```

Comparando duas strings

- Da mesma maneira como o operador de atribuição não funciona para strings, o mesmo ocorre com operadores relacionais usados para comparar duas strings. Desse modo, para saber se duas strings são iguais usa-se a função `strcmp()`:

```
int strcmp(const char *str1, const char *str2);
```

- A função `strcmp()` compara posição a posição as duas strings (`str1` e `str2`) e retorna um valor inteiro igual a zero, no caso de as duas strings serem iguais. Um valor de retorno diferente de zero significa que as strings são diferentes.

Exemplo com a função strcmp()

```
1 #include <stdio.h> // prog200.c
2 #include <string.h>
3
4 int main() {
5     char str1[30], str2[30];
6
7     printf("digite a primeira string (max 29 chars): ");
8     fgets(str1, 30, stdin);
9     printf("digite a segunda string (max 29 chars): ");
10    fgets(str2, 30, stdin);
11
12    if(strcmp(str1, str2) == 0) {
13        printf("strings iguais\n");
14    } else {
15        printf("strings diferentes\n");
16    }
17
18    return 0;
19 }
```

Concatenando duas strings

- A **concatenação** consiste em copiar uma string para o final de outra string.

Concatenando duas strings

- A **concatenação** consiste em copiar uma string para o final de outra string.
- Na linguagem C, para fazer a concatenação de duas strings, usa-se a função `strcat()`:

```
char* strcat(char *destino, const char *origem);
```

Concatenando duas strings

- A **concatenação** consiste em copiar uma string para o final de outra string.
- Na linguagem C, para fazer a concatenação de duas strings, usa-se a função `strcat()`:

```
char* strcat(char *destino, const char *origem);
```

- Basicamente, a função `strcat()` copia a sequência de caracteres contida em origem para o final da string destino e retorna o endereço da string destino.

Concatenando duas strings

- A **concatenação** consiste em copiar uma string para o final de outra string.
- Na linguagem C, para fazer a concatenação de duas strings, usa-se a função `strcat()`:

```
char* strcat(char *destino, const char *origem);
```

- Basicamente, a função `strcat()` copia a sequência de caracteres contida em origem para o final da string destino e retorna o endereço da string destino.
- O primeiro caractere da string contida em origem é colocado no lugar do caractere `'\0'` da string destino.

Exemplo com a função strcat()

```
1 #include <stdio.h> // prog201.c
2 #include <string.h>
3
4 int main() {
5     char destino[30] = "bom ";
6     char origem[30] = "dia";
7
8     strcat(destino, origem);
9
10    printf("%s\n", destino);
11
12    return 0;
13 }
```


Biblioteca `ctype.h`

`<ctype.h>`: útil ao se trabalhar com caracteres.

<code>int isalnum(int c)</code>	Verifica se <code>c</code> é um dígito decimal ou uma letra maiúscula ou minúscula.
<code>int isalpha(int c)</code>	Verifica se <code>c</code> é uma letra alfabética.
<code>int isblank(int c)</code>	Verifica se <code>c</code> é um caractere em branco (espaço ou tab)
<code>int isdigit(int c)</code>	Verifica se <code>c</code> é um caractere de dígito decimal.
<code>int islower(int c)</code>	Verifica se <code>c</code> é uma letra minúscula.
<code>int isupper(int c)</code>	Verifica se o parâmetro <code>c</code> é uma letra alfabética maiúscula.
<code>int isspace(int c)</code>	Verifica se <code>c</code> é um caractere de espaço em branco, ou seja, ' ' (space), '\t' (horizontal tab), '\n' (newline), '\v' (vertical tab), '\f' (feed), '\r' (carriage return)

Exemplo — ctype.h

```
1 #include <stdio.h> // prog347.c
2 #include <ctype.h>
3
4 int main() {
5     char ch;
6     printf("Digite um caractere: ");
7
8     while((ch = getchar()) != EOF) {
9         printf("o caractere \'%c\': ", ch);
10        if(isalnum(ch)) printf("\né alfanumérico");
11        else printf("\nnão é alfanumérico");
12        if(isdigit(ch)) printf("\né um dígito decimal");
13        else printf("\nnão é um dígito decimal");
14        if(isspace(ch)) printf("\né um espaço");
15        else printf("\nnão é um espaço");
16        if(islower(ch)) printf("\neh minúsculo");
17        else printf("\nnão é minúsculo");
18        if(isupper(ch)) printf("\né maiúsculo");
19        else printf("\nnão é maiúsculo");
20        printf("\nDigite um caractere: ");
21        getchar(); // limpa buffer
22    }
23 }
```

Convertendo strings para números



Conversão de string para tipos nativos

Função	Tipo retornado
<code>int atoi(const char *nptr);</code>	<code>int</code>
<code>long atol(const char *nptr);</code>	<code>long</code>
<code>long long atoll(const char *nptr);</code>	<code>long long</code>
<code>double atof(const char *nptr);</code>	<code>double</code>

Exemplo: conversão de string para numérico

```
1 #include <stdio.h> // prog120a.c
2 #include <stdlib.h>
3
4 int main(void) {
5     char b1[10] = "123";
6     char b2[10] = "123456789";
7     char b3[10] = "12.345";
8     char b4[17] = "2.546312891234";
9
10    int a = atoi(b1);
11    long b = atol(b2);
12    float c = (float) atof(b3);
13    double d = atof(b4);
14
15    printf("%d %ld %f %.12f\n", a, b, c, d);
16
17    return 0;
18 }
```

Conversão de tipos nativos para string

- Na linguagem C, é possível converter de tipos nativos para `char*`
- Para isso, existe a função `snprintf()`:

```
int snprintf(char *str, size_t size, const char *format, ...);
```

- Escreve no máximo `size-1` caracteres em `str` (mais o `'\0'` no fim). Retorna o número de caracteres que teria escrito (sem contar o `'\0'`).

Exemplo: conversão de numérico para string

```
1 #include <stdio.h> // prog120b.c
2
3 int main(void) {
4     char buf[20];
5     int a = 255;
6     long t = 123456789;
7     float b = 347.561987;
8     double c = 569.78961234;
9
10    snprintf(buf, sizeof(buf), "%d", a);
11    printf("buf = %s\n", buf);
12
13    snprintf(buf, sizeof(buf), "%ld", t);
14    printf("buf = %s\n", buf);
15
16    snprintf(buf, sizeof(buf), "%.2f", b);
17    printf("buf = %s\n", buf);
18
19    snprintf(buf, sizeof(buf), "%.8f", c);
20    printf("buf = %s\n", buf);
21
22    return 0;
23 }
```

Exercícios

1. Sem usar a função `strlen()`, faça um programa que leia uma string e imprima quantos caracteres ela possui.
2. Faça um programa que leia uma string e a imprima de trás para a frente.

1. Faça um programa que leia uma string e imprima uma mensagem dizendo se ela é um palíndromo ou não. Um **palíndromo** é uma palavra que tem a propriedade de poder ser lida tanto da direita para a esquerda como da esquerda para a direita. Exemplos: ovo, arara, rever, asa, osso etc.

Exercícios

1. Escreva um programa que leia uma string do teclado e converta todos os seus caracteres em maiúscula. Dica: subtraia 32 dos caracteres cujo código ASCII está entre 97 e 122.
2. Escreva um programa que leia uma string do teclado e converta todos os seus caracteres em minúscula. Dica: some 32 dos caracteres cujo código ASCII está entre 65 e 90.

1. O código de César é uma das técnicas de criptografia mais simples e conhecidas. É um tipo de substituição no qual cada letra do texto é substituída por outra, que se apresenta n posições após ela no alfabeto. Por exemplo, com uma troca de três posições, a letra *A* seria substituída por *D*, *B* se tornaria *E* e assim por diante. Escreva um programa que faça uso desse código de César para três posições. Entre com uma string e imprima a string codificada.

Exemplo:

- String: a ligeira raposa marrom saltou sobre o cachorro cansado.
- Nova string: d oljhlud udsrvd pduurp vdowrx vreuh r fd ruur fdqvdr

Exercícios

1. Escreva um programa que leia duas strings e as imprima em ordem alfabética, a ordem em que elas apareceriam em um dicionário.

FIM

